

Managing Storage (partitioning):

Types of Storage :

1. DAS (Direct Attached Storage)
2. SAN (Storage Area Network)
3. NAS (Network Attached Storage)

Working With DAS:

Disk that is not partitioned called RAW or Unallocated

Partitioning Procedure:

1. Make partitions (through utilities like fdisk, gdisk and parted)
2. Apply FileSystem (mkfs.[fs] or mkfs -t [fs])
3. Mount those Partitions (mount utility)

Displaying Block Devices :

1. Command 'lsblk'
2. Command 'cat /proc/partitions'

Partitioning:

What's the partition?

The partition is just a part of a disk that's has start and end .

Naming of disks:

- The kernel that implement and use these names before the kernel started these names aren't in use yet.

Names of Disks depend on the type of disk like:

PATA disks or as we called it Data Disks >>> hd

SATA – Scsi – ssd disks >>> sd

Virtual Disks >> vd

These disks are alphabetically ordered for example :

1st disk is called sda , 2nd disk is sdb , 3rd disk is sdc etc.

Partitions is numerically ordered for example sdc1 means that's the 1st partitions in the 3rd disk .

Question: Is sdc7 means that's the 7th partition in the 3rd disk?

Answer: it's depend on what is the scheme you use for partitioning, if you use MBR the number of the partition is different if you use the GPT.

MBR vs GPT

MBR as Partitioning scheme:

Features of MBR

- MBR stand for Master Boot Record
- MBR is invented By Microsoft with the invention of personal computer
- MBR placed in first 512 byte from the disk
- MBR is 512 byte as :
 - 64 byte partition table
 - 446 byte boot loader
 - 2 byte magic number (MBR signature)
- MBR supports a maximum of four primary partitions and (max of 63 logical partitions) , with a total size of the hard disk of 2Tbyte (2^{31} byte)

As you notice all features of MBR is not suitable for storage that you use today .

Types of partitions: Primary – extended – logical

Difference between extended and the rest is the extended is a container to create logical partitions only and there's no file system applied on it .

Logical partitions starts from sdb5 (example) the rest of 1-2-3-4 is reserved for primary even if you didn't create any of them.

Examples:

Sdb1 is 1st primary and sdb7 is 2nd logical partition.

GPT

Features :

- Stands for GUID partition table
- Max number of Partitions 128
- Placed in first (primary) and last (secondary/backup) sectors of a disk
- Max size of disk 8 zebi byte (zebi byte = $1024 \times 1024 \times 1024 \times 1024$ giga byte)

Storage Measurement Units

Symbol	Name	Value	Symbol	Name	Value
KB	Kilobyte	1000^1	KiB	Kibibyte	1024^1
MB	Megabyte	1000^2	MiB	Mebibyte	1024^2
GB	Gigabyte	1000^3	GiB	Gibibyte	1024^3
TB	Terabyte	1000^4	TiB	Tebibyte	1024^4
PB	Petabyte	1000^5	PiB	Pebibyte	1024^5
EB	Exabyte	1000^6	EiB	Exbibyte	1024^6
ZB	Zettabyte	1000^7	ZiB	Zebibyte	1024^7
YB	Yottabyte	1000^8	YiB	Yobibyte	1024^8

Creating MBR Partitions with fdisk

1. Open a root shell and run the **fdisk** command. This command needs the name of the disk device where you want to create the partition as its argument. In this exercise I'll use `/dev/vda`. Change if needed according to your hardware.

```
[root@localhost ~]# fdisk /dev/vda
```

Welcome to fdisk (util-linux 2.23.2).

Changes will remain in memory only until you decide to write them. Be careful before using the **write** command.

2. Before doing anything, it is a good idea to check how much disk space you have available. Press **p** to see an overview of current disk allocation.

Command (m for help): p

Disk /dev/vda: 6442 MB, 6442450944 bytes, **12582912** sectors

Units = sectors of 1 * 512 = 512 bytes

Sector size (logical/physical): 512 bytes / 512 bytes

I/O size (minimum/optimal): 512 bytes / 512 bytes

Disk label type: dos

Disk identifier: 0x000a056b

Device Boot Start End Blocks Id System

/dev/vda1 * 2048 514047 256000 83 Linux

/dev/vda2 514048 **8984575** 4235264 8e Linux LVM

Chapter 14: Managing Partitions 327

In the output of this command, in particular look for the total number of sectors and the last sector that is currently used (marked in bold in the above command output). If the last partition does not end on the last sector, you have available space to create a new partition.

3. Type **n** to add a new partition.

Command (m for help): n

Partition type:

p primary (2 primary, 0 extended, 2 free)

e extended

Select (default p):

4. Assuming you have a `/dev/vda1` and a `/dev/vda2` partition and nothing else, select **p** to create a primary partition. Accept the partition number that is now suggested.

5. Specify the first sector on disk that the new partition will start on. The first available sector is suggested by default, press Enter to accept.

6. Specify the last sector that the partition will end on. By default, the last sector available on disk is suggested. If you use that, after this exercise you will not have any disk space left to create additional partitions or logical volumes, so you should use another last sector. To use another last sector, you can do the following:

- Enter the number of the last sector you want to use.
- Enter **+number** to create a partition that sizes a specific number of sectors.
- Enter **+number(K,M,G)** to specify the size you want to assign to the partition in KiB, MiB, or GiB.
- Type **+100M** to make this a 100 MiB partition.

Partition number (3,4, default 3):

First sector (8984576-12582911, default 8984576):

Using default value 8984576

Last sector, +sectors or +size{K,M,G} (8984576-12582911, default 12582911): +100M

Partition 3 of type Linux and of size 100 MiB is set

After you enter the partition's ending boundary, fdisk will show a confirmation.

7. At this point, you can define the partition type. By default, a Linux partition type is used. If you want the partition to be of any other partition type, use **t** to change it. Common partition types include the following:

328 Red Hat RHCSA/RHCE 7 Cert Guide

- 82: Linux swap
- 83: Linux
- 8e: Linux LVM

Press **Enter** to accept the default Linux partition type **83**.

8. If you are happy with the modifications, press **w** to write them to disk and exit fdisk. If you have created a partition on a disk that is already in use, you now see the following message:

Command (m for help): w

The partition table has been altered!

Calling ioctl() to re-read partition table.

WARNING: Re-reading the partition table failed with error 16: Device or resource busy. The kernel still uses the old table. The new table will be used at the next reboot or after you run partprobe(8) or kpartx(8) Syncing disks.

[root@localhost ~]#

9. This message indicates that the partition has successfully been added to the partition table, but the in-memory kernel partition table could not be updated. You can see that by comparing the output of **fdisk -l /dev/vda** with the output of the command **cat /proc/partitions**, which shows the kernel partition table.

10. Type **partprobe /dev/vda** to write the changes to the kernel partition table. The partition has now been added and you can create a file system .

HINT: You see the “re-reading the partition table failed with error 16” message only if you are adding partitions to a disk that already has some mounted partitions. If you are working on a new disk that does not have any mounted partitions, you will not see this error and you will not have to use the **partprobe** command.

Command dd:

dd is a command-line utility for Unix and Unix-like operating systems whose primary purpose is to convert and copy files.

Destroy MBR : to make the disk unallocated again and destroy MBR

```
#dd if=/dev/zero of=/dev/sdb bs=1M count=1
```

To backup the entire harddisk

```
# dd if = /dev/sda of = /dev/sdb
```

To backup a Partition :

```
# dd if=/dev/hda1 of=~ /partition.img
```

To create an image of a Hard Disk

```
# dd if = /dev/hda of = ~/hdadisk.img
```

To restore using the Hard Disk Image :

```
# dd if = hdadisk.img of = /dev/hdb
```

Creating GPT Partitions with gdisk

1. To create a partition with gdisk, type **gdisk /dev/vdb**

```
[[root@localhost ~]# gdisk /dev/vdb
```

2. Type **n** to enter a new partition.
3. You now are asked to enter the first sector. By default, the first sector that is available on disk will be used, but you can specify an offset as well. This does not make sense at all, so just press **Enter** to accept the default first sector that is proposed.
4. When asked for the last sector, by default the last sector that is available on disk is proposed (which would create a partition that fills the entire hard disk). You can specify a different last sector, or specify the disk size using **+**, the size, and KMGTP. So to create a 2 TiB disk partition, use **+2TiB**.

5. You now are asked to set the partition type. If you do not do anything, the partition type is set to 8300, which is the Linux file system partition type.
6. The partition is now created (but not yet written to disk). Press **p** to show an overview
7. If you are satisfied with the current partitioning, press **w** to write changes to disk and commit.
8. If at this point you get an error message indicating that the partition table is in use, type **partprobe** to update the kernel partition table.

Create File System

```
[root@master ~]# mkfs.ext4 /dev/sdb1
```

Mount the Partition

```
[root@master ~]# mkdir /mydata
```

```
[root@master ~]# mount /dev/sdb1 /mydata/
```

List block devices, mounted and file system

```
[root@master ~]# df -hT
```

```
[root@master ~]# lsblk -f
```

Eng. Muhammad Adel